Family Planner — Development Actions & System Reference

Generated: May 2026

System Overview

Family Planner is a local-first household management web application running as a Docker Compose stack on a home server at **192.168.10.14**. It provides task management, budgeting, tax receipt processing, and daily roster scheduling for a family.

Technology Stack

Layer	Technology
Frontend	React 18, TypeScript, Vite, lucide-react, recharts
Backend	FastAPI, Python 3.12, uvicorn, Pydantic v2
Database	SQLite (multiple databases per domain)
Container	Docker Compose, nginx reverse proxy
Source control	GitHub

Directory Structure

```
app/
  frontend/src/
    main.tsx          — App shell, routing, sidebar nav, profile selector
    types.ts          — All TypeScript types, constants, helper functions
    styles.css        — Single CSS file, custom properties, no framework
    pages/
      Dashboard.tsx
      FamilyBudget.tsx
      TaxReceipts.tsx
      Todo.tsx         — TodoPage + TaskManagementPage + shared components
      Settings.tsx     — All settings sections (general, users, backup, etc.)
  backend/app/
    main.py           — FastAPI routes
    models.py         — Pydantic models
    db.py             — SQLite helpers, DEFAULT_PAGE_PERMISSIONS
  nginx/              — Reverse proxy config
```

Data Architecture

SQLite Databases (in backend /data/ volume)

File	Purpose
`family_budget.sqlite3`	Budget items, savings, categories, actuals, **user profiles**
`tasks.sqlite3`	Tasks and subtasks
`roster.sqlite3`	Daily roster schedules
`receipt_drafts.sqlite3`	OCR/AI draft state per SharePoint file

assigned_to Field (Tasks & Subtasks)

Both tasks and subtasks store assignees as a **JSON array string** in SQLite:

- `["everyone"]` — visible/assigned to all family members
- `["uuid-1", "uuid-2"]` — specific profiles by ID

Python helpers in db.py:

- `_parse_assigned_to(raw)` → `list[str]` — handles legacy string, JSON array, or None
- `_serialise_assigned_to(val)` → JSON string — normalises list/string for storage

User Profiles

Stored in `family_budget.sqlite3 → user_profiles` table.

`DEFAULT_PAGE_PERMISSIONS` in `db.py` is the authoritative allowlist of valid page permission keys. Any permission not in this list is **silently stripped on save** — new pages must be added here.

Roles:

- **Administrator** — full access, sees all tasks
- **User** — sees only tasks assigned to them or 'everyone'

Key Development Actions

1. Multi-Assignee Support for Tasks & Subtasks

Problem: `assigned_to` was a single string; family members couldn't share tasks.

Changes:

- Migrated `Task.assigned_to` and `SubTask.assigned_to` from `string` to `string[]` in TypeScript types
- Added `_parse_assigned_to` / `_serialise_assigned_to` helpers in `db.py`
- Added `ALTER TABLE subtasks ADD COLUMN assigned_to TEXT NOT NULL DEFAULT '["everyone"]'` migration
- Built `AssigneeToggle` React component: button-group multi-select showing all profiles + Everyone
- `assigneeLabel()` helper renders names inline (e.g. `Isaac, Max` or `Everyone`)

AssigneeToggle behaviour:

- Selecting 'Everyone' clears all individual selections
- Deselecting all individuals reverts to Everyone
- Compact mode shows abbreviated labels for tight layouts

---

2. Task Management Page — Two-Column Layout

Problem: Subtask editing was in a separate right-panel, disconnected from task editing.

Changes:

- Removed standalone detail panel from Task Management page
- Edit form redesigned as a **two-column grid**: task fields left (50%), subtask manager right (50%)
- `TaskDetailPanel` given an `embedded` prop — when true, renders only the subtask list without header/close button
- Added `AssigneeToggle` to both task edit form and subtask add/edit form

---

3. To Do Page — Role-Based Task Filtering

Problem: All users could see all tasks regardless of assignment.

Changes:

- `TodoPage` now checks `activeProfile.role`
- **Administrator:** sees all non-template tasks
- **User:** sees only tasks where `assigned_to` includes their profile ID or `'everyone'`

```
const isAdmin = activeProfile?.role === 'Administrator';
const instances = tasks.filter(t => {
  if (t.is_template) return false;
  if (isAdmin) return true;
  const ids = Array.isArray(t.assigned_to) ? t.assigned_to : [t.assigned_to];
  return ids.includes('everyone') || ids.includes(activeProfile?.id ?? '');
});
```

4. Auto-Assign Parent Task from Subtask

Problem: Assigning a subtask to someone didn't update the parent task's assignees.

Root Causes (fixed in sequence):

1. **Stale React closure** — `assignSubtask` read from `tasks` state captured at render time, not current DB state. Fixed by making a fresh `GET /api/tasks` call.
2. **Save overwrites auto-assign** — `saveEditTask` sent the entire `editingTask` object including the old `assigned_to`, overwriting the DB update. Fixed by calling `setEditingTask(v => v && v.id === taskId ? { ...v, assigned_to: merged } : v)` to sync React state immediately after the patch.

Final logic:

- When a subtask is assigned to specific people, fetch fresh task data
- Merge new assignees into parent task's existing assignees (no duplicates)
- If parent was 'everyone', replace with the specific assignees
- Update both DB and React state before calling `load()`

---

5. Settings — General Sub-Page

Problem: Theme was buried in the SharePoint settings section; no timezone setting existed; Backup & Restore was a standalone nav item.

Changes:

- Added `'settings-general'` to `Page` type union in `types.ts`
- Added `'settings-general': 'General'` to `pageLabels`
- Added `timezone: string` to `SettingsState` (default: `'Australia/Brisbane'`)
- Added `timezone: 'Australia/Brisbane'` to `defaultSettings`
- New **General** section in `Settings.tsx` containing:
  - **Appearance** card — theme toggle (light / dark / system)
  - **Regional** card — timezone dropdown with AU zones + common international
  - **Portability** card — Backup download and Restore upload
- Removed theme card from SharePoint section
- Removed Backup & Restore from sidebar nav (now lives on General page)
- Added General as first item under Settings in sidebar

---

6. Permission System Fix — settings-general

Problem: Saving a user profile with the General permission ticked caused it to be silently dropped.

Root cause: `DEFAULT_PAGE_PERMISSIONS` in `db.py` is a hardcoded allowlist. Any permission key not present is stripped on save. `'settings-general'` was not in the list.

Fix: Added `'settings-general'` to `DEFAULT_PAGE_PERMISSIONS` in `db.py` and rebuilt the backend container.

Lesson: Every new page added to the app must also be added to `DEFAULT_PAGE_PERMISSIONS`.

7. PIN Screen — Hide Profile Tiles During PIN Entry

Problem: Profile tiles remained visible while a PIN was being entered, cluttering the screen.

Fix: Wrapped profile tile grid in `{!pendingProfile && ...}` in `ProfileSelectScreen` in `main.tsx`. Tiles disappear as soon as a profile is selected and PIN entry begins; they reappear on cancel or after authentication.

---

8. Subtask Assignee Labels in Task Lists

Problem: No visual indication of who a subtask was assigned to when viewing task lists.

Fix: Added inline assignee label after each subtask title in `TaskRow`:

```
<span style={{ fontSize: '0.72rem', opacity: 0.55, marginLeft: '0.35rem' }}>
  {' — ' + assigneeLabel(st.assigned_to?.length ? st.assigned_to : ['everyone'],
userProfiles)}
</span>
```

Appears on both the Task Management page and the To Do page, but NOT in the subtask edit form.

---

9. Backend Bug Fixes

Bug	Fix
`create_subtask` silently failing (HTTP 200 with error body)	Function signature used wrong variable name `data` vs `patch`
`update_task` not serialising `assigned_to`	Referenced undefined `patch` instead of `data`
`ALTER TABLE` syntax error in db.py	Python string contained unescaped single quotes inside f-string
502 errors after rebuild	Python syntax error in db.py prevented backend from starting

Development Workflow

Standard Build & Deploy

```
# SSH to server
ssh utilities@192.168.10.14
cd /home/utilities/.openclaw/workspace/Family-Planner

# Build frontend first (catches TypeScript errors cheaply)
cd app/frontend && npx vite build && cd ../..

# Rebuild and restart containers
docker compose up --build -d

# Verify
curl http://localhost/api/health
docker compose logs -f backend
```

Applying Code Changes

For complex multi-line changes (especially JSX/TSX), the most reliable approach is:

1. Write a Python script locally with exact `str.replace()` operations
2. SCP the script to the server: `scp fix.py utilities@192.168.10.14:/tmp/`
3. Run it: `ssh utilities@192.168.10.14 "python3 /tmp/fix.py"`
4. Rebuild frontend to check for TypeScript errors before Docker rebuild

Why not heredoc/sed? JSX contains characters (`<` , `>` , `{` , `}` , `'` , `"`) that interfere with shell quoting and sed patterns at scale. Python `str.replace()` with exact string literals is safer.

Troubleshooting

Symptom	First check
502 Bad Gateway	`docker compose logs backend` — usually a Python syntax error
Blank page	Browser console — usually a TypeScript/React runtime error
API returns `{"status":"failed"}` with HTTP 200	Backend function has a Python NameError or similar
Permission not saving	Check `DEFAULT_PAGE_PERMISSIONS` in db.py
Auto-assign not working	Check for stale React closure — use fresh API fetch

---

API Endpoints Reference

Tasks

```
GET    /api/tasks                      → { tasks: Task[] }
POST   /api/tasks                      body: TaskCreate → Task
```

```
PATCH  /api/tasks/{task_id}              body: partial Task fields → ActionResponse
DELETE /api/tasks/{task_id}              → ActionResponse
```

Subtasks

```
GET    /api/subtasks/{task_id}           → { subtasks: SubTask[] }
POST   /api/subtasks/{task_id}           body: { title, assigned_to? } → SubTask
PATCH  /api/subtasks/{subtask_id}        body: partial SubTask fields → ActionResponse
DELETE /api/subtasks/{subtask_id}        → ActionResponse
```

User Profiles

```
GET    /api/profiles                     → { profiles: UserProfile[] }
POST   /api/profiles                     body: ProfileCreate → UserProfile
PATCH  /api/profiles/{profile_id}        body: partial profile → ActionResponse
DELETE /api/profiles/{profile_id}        → ActionResponse
```

Backup

```
GET    /api/backup/download              → zip file stream
POST   /api/backup/restore              body: multipart/form-data (zip) → ActionResponse
```